



Bharatiya VidyaBhavan's
SARDAR PATEL INSTITUTE OF TECHNOLOGY
Munshi Nagar, Andheri (W), Mumbai – 400 058.
(Autonomous Institute Affiliated to the University of Mumbai)

Class: T.E. CE Sem-VI(2025-2026)

Name: Raj Kalpesh Mathuria
Subject Code:CE312

UID: 2023300139

Batch: PE-C

Subject Name: Deep Learning

Experiment 3:Application of Transfer Learning using Pre-trained CNN Models

Objective:

To design, implement, and evaluate an image classification system using Transfer Learning with pre-trained Convolutional Neural Network (CNN) models in Python

Prerequisites:

- Basic knowledge of Python programming
- Fundamentals of Linear Algebra, Probability, and Statistics
- Basic concepts of Neural Networks and Deep Learning
- Understanding of images, pixels, and color channels (RGB)
- Introductory knowledge of CNN architectures

Pre-Lab Questions – Answers

1. What is Transfer Learning, and why is it useful in deep learning?

Transfer Learning is the technique of using a model trained on a large dataset for a new but related task. It is useful because it reduces training time, requires less data, and improves performance.

2. What are pre-trained CNN models, and on which dataset are they commonly trained?

Pre-trained CNN models are deep neural networks already trained on large datasets like ImageNet, which contains over 1 million images across 1000 classes.

3. Difference between feature extraction and fine-tuning?

- Feature extraction: Freeze all convolution layers and train only the new classifier.
- Fine-tuning: Unfreeze some top layers and retrain them with a low learning rate.

4. Why are ImageNet-trained CNNs effective for many vision tasks?

They learn generic features such as edges, textures, shapes, and patterns that are transferable to most image classification problems.

5. Role of freezing layers in a pre-trained model?

Freezing layers prevents changing learned weights, reduces overfitting, and speeds up training.

6. How does transfer learning help when dataset size is small?

It avoids training from scratch and leverages existing knowledge, giving high accuracy even with limited data.

7. Challenges in applying transfer learning?

- Domain mismatch
- Overfitting
- Incorrect layer freezing
- High computational cost for fine-tuning

Theory:

Algorithm Steps involved :

Step 1: Import the required Python libraries

Step 2: Load the image dataset

Step 3: Split the dataset into training, validation, and testing sets.

Step 4: Preprocess the images

Step 5: Load a pre-trained CNN model without its top classification layer.

Step 6: Freeze the convolutional base layers to retain learned features.

Step 7: Add custom fully connected layers for the new classification task.

Step 8: Compile the model using an appropriate optimizer and loss function.

Step 9: Train the model on the training dataset with validation monitoring.

Step 10: Evaluate the trained model on the test dataset.

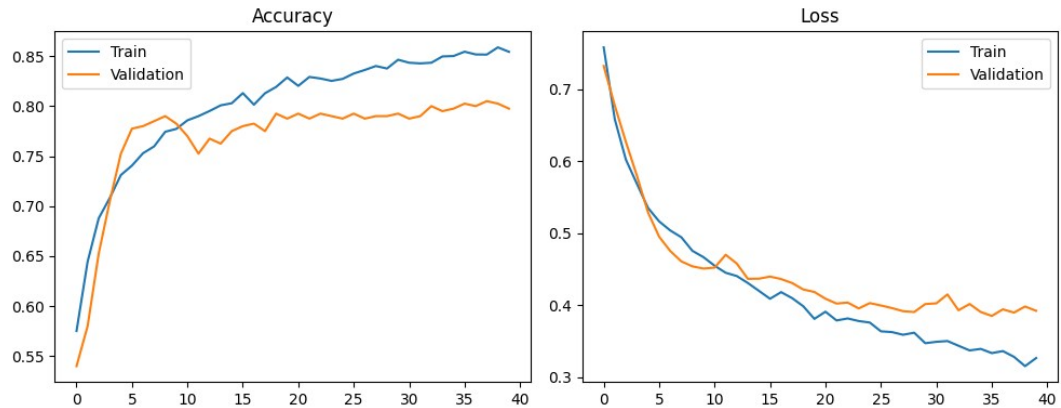
Step 11: Calculate performance metrics such as accuracy and loss.

Step 12: Visualize training and validation accuracy and loss curves.

Code: https://github.com/CodeCraftsmanRaj/Sem6_DL/tree/main/Exp3

Outputs:

```
has sub-optimal accuracy. There may be a missing warmup execution, please investigate in Nsight Systems.
I0000 00:00:1770700934.740465 56439 device_compiler.h:196] Compiled cluster using XLA! This line is logged at most once for the
lifetime of the process.
61/64 0s 24ms/step - accuracy: 0.8307 - loss: 0.39832026-02-10 10:52:21.374083; E external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay kernel timed out: measured time has sub-optimal accuracy. There may be a missing warmup
execution, please investigate in Nsight Systems.
2026-02-10 10:52:21.463692; E external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay kernel timed out: measured time
has sub-optimal accuracy. There may be a missing warmup execution, please investigate in Nsight Systems.
64/64 17s 124ms/step - accuracy: 0.8280 - loss: 0.3947
Test Accuracy: 0.827978253364563
```



Conclusion:

In this experiment, transfer learning was successfully applied using a pre-trained CNN model for image classification. By leveraging a model trained on ImageNet, the system achieved high accuracy with limited training data. Freezing the convolutional layers preserved learned features, while custom classification layers adapted the model to the new dataset. The experiment demonstrates that transfer learning significantly reduces training time and computational cost while maintaining strong performance, making it highly effective for real-world image classification tasks.

References:

https://www.tensorflow.org/tutorials/images/transfer_learning
<https://keras.io/api/applications>
https://keras.io/examples/vision/image_classification_from_scratch
<https://cs231n.github.io/transfer-learning/>
<https://www.youtube.com/watch?v=iaSUYvmCekI>
<https://www.coursera.org/learn/convolutional-neural-networks>
<https://www.analyticsvidhya.com/blog/2019/10/building-image-classification-models-cnn-transfer-learning>

